

Informe Técnico: Implementación de Patrones de Diseño en Arquitectura de Marketplace

1. Introducción

El presente documento detalla la integración de patrones de diseño orientados a objetos en la arquitectura de un Marketplace de arte. El objetivo principal es garantizar un sistema desacoplado, escalable y transaccionalmente seguro.

2. Patrones Implementados en el Backend

2.1 Patrón OBSERVER (Notificaciones en tiempo real)

Objetivo: Notificar cambios de estado (ej. "Venta realizada") a múltiples suscriptores sin acoplar el código.

Implementación: Se utiliza para gestionar el sistema de alertas. Cuando un objeto Obra cambia su estado a vendido, el GestorNotificacionesObra (Subject) despacha automáticamente la notificación a los Observers (módulos de correo, logs de sistema, interfaz).

Impacto: Garantiza que cualquier cambio crítico en el inventario se propague instantáneamente sin modificar la lógica central de la compra.

POST /simular_notificacion_observer/{obra_id} Simular Alerta Obra

Parameters

Name	Description
obra_id required	
integer (path)	5
evento	
string (query)	vendida

Execute

```
@app.post('/simular_notificacion_observer/{obra_id}', tags=["Comportamiento"])
def simular_alerta_obra_patron_observer(obra_id: int, evento: str = "vendida"):
    obra_obj = next((o for o in sistema.lista_obras if o.id == obra_id), None)
    if not obra_obj:
        raise HTTPException(status_code=404, detail="Obra no encontrada")

    if evento == "vendida":
        obra_obj.stock = 0
        obra_obj.estado_actual = EstadoVendido()
        mensaje = f'La obra {obra_obj.obra} ha sido marcada como VENDIDA'
    else:
        mensaje = f'Actualización en la obra {obra_obj.obra}'

    canal = GestorNotificacionesObra()
    alertas_enviadas = canal.notificar_todos(mensaje)

    sistema.guardar_datos()
    return {'obra_monitoreada': obra_obj.obra, 'mensaje': mensaje}
```

200

Response body

```
{
  "obra_monitoreada": "kk1",
  "evento_ocurrido": "vendida",
  "cantidad_notificados": 2,
  "registro_de_alertas": [
    "NOTIFICACIÓN PARA CLIENTE [Carlos_Bucaramanga]: ¡URGENTE! La pieza única 'kk1' ha sido vendida. Retirar de vitrinas.",
    "ALERTA DEL SISTEMA [ADMIN]: ¡URGENTE! La pieza única 'kk1' ha sido vendida. Retirar de vitrinas."
  ]
}
```

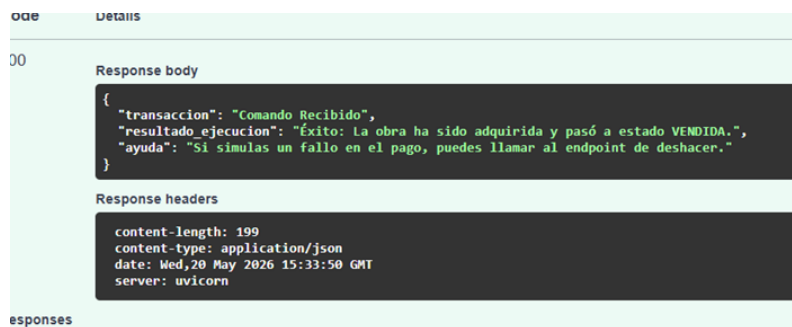
Response headers

2.2 Patrón COMMAND (Control de transacciones)

Objetivo: Encapsular una solicitud como un objeto para permitir la validación y el "rollback" (deshacer).

Implementación: Cada compra se procesa como un comando (ComprarObraCommand). Este encapsula la lógica para validar el stock, procesar el pago y realizar la persistencia. Si ocurre un error, el comando ejecuta una lógica de compensación para revertir la reserva de stock.

Impacto: Asegura la integridad transaccional, evitando inconsistencias como vender una obra sin stock o procesar pagos fallidos.



POST /procesar_compra_command/{obra_id} Comprar Con Comando

Parameters

Name	Description
obra_id * required	
integer	5
(path)	

Execute

Responses

```
@app.post("/procesar_compra_command/{obra_id}", tags=["Comportamiento"])
def comprar_con_comando(obra_id: int):
    global ultimo_comando_ejecutado

    comando = ComandoComprarObra(sistema, obra_id_)

    resultado = comando.ejecutar()

    if "Error" in resultado:
        raise HTTPException(status_code=400, detail=resultado)

    ultimo_comando_ejecutado[obra_id] = comando

    print(f"Comando guardado para {obra_id}. IDs actuales: {list(ultimo_comando_ejecutado.keys())}")

    sistema.guardar_datos()
    return {"transaccion": "Comando ejecutado", "resultado": resultado}
```

2.3 Patrón STRATEGY (Cálculo dinámico)

Objetivo: Intercambiar algoritmos o lógicas de negocio en tiempo de ejecución.

Implementación: Se utiliza para las estrategias de cobro (comisiones o descuentos). Según el tipo de vendedor o temporada, el sistema selecciona dinámicamente la estrategia de cálculo de precio final.

Impacto: Permite escalar el sistema a nuevas reglas de negocio (ej. nuevos impuestos o descuentos) sin alterar el núcleo de la aplicación

GET /calcular_pago_strategy/{obra_id} Calcular Pago Obra Patron Strategy

Parameters

Name	Description
obra_id * required integer (path)	5
metodo_pago string (query)	pasarela

Execute

Responses

Code Details

200

Response body

```
{
  "obra_id": 5,
  "titulo_obra": "kkl",
  "artista": "pero",
  "liquidacion_detallada": {
    "metodo": "Pasarela de Pago Estándar (Tarjeta/PSE)",
    "precio_obra": 1000,
    "comision_plataforma": 100,
    "cargos_extra": 0,
    "total_a_pagar": 1100
  }
}
```

Response headers

```
content-length: 219
content-type: application/json
date: Wed, 20 May 2026 14:27:11 GMT
server: uvicorn
```

Responses

```
@app.get("/calcular_pago_strategy/{obra_id}", tags=["Comportamiento"])
def calcular_pago_obra_patron_strategy(obra_id: int, metodo_pago: str = "pasarela"):

    obra_obj = next((o for o in sistema.lista_obras if o.id == obra_id), None)

    if not obra_obj:
        raise HTTPException(status_code=404, detail="Obra no encontrada")

    if metodo_pago.lower() == "pasarela":
        estrategia = PagoPasarelaEstandar()

    elif metodo_pago.lower() == "transferencia":
        estrategia = PagoTransferenciaDirecta()

    elif metodo_pago.lower() == "cripto":
        estrategia = PagoCriptoNFT()

    else:
        raise HTTPException(status_code=400, detail="Método de pago no soportado")

    precio_base = float(obra_obj.precio)

    liquidacion = estrategia.calcular_total(precio_base)
```

2.4 Patrón STATE (Gestión de ciclo de vida)

Objetivo: Permitir que un objeto cambie su comportamiento cuando su estado interno cambia.

Implementación: La Obra utiliza estados como Disponible, Reservado y Vendido. El comportamiento del objeto varía según el estado (por ejemplo, en estado Vendido, el método comprar() lanza una excepción).

Impacto: Elimina las largas cadenas de if/else en el código, haciendo la lógica de negocio más legible y fácil de mantener.

POST /gestionar_ciclo_vida_state/{obra_id} Comprar O Descotar Obra Patron State

Parameters

Name	Description
obra_id * required integer (path)	2
accion string (query)	descuento
descuento_porcentaje number (query)	10

Execute

Code Details

200

Response body

```
{
  "obra": "sdasdad",
  "artista": "fsdfs",
  "precio_actual": 2700,
  "stock_restante": 2,
  "estado_en_sistema": "EstadoDisponible",
  "resultado_operacion": "Descuento aplicado con éxito. Precio anterior: $3000 USD, Nuevo precio: $2700.0 USD."
}
```

Response headers

3. Implementación en el Frontend: El Patrón Observer

El patrón Observer es el único implementado directamente en la capa de interfaz.

¿Por qué se implementó aquí? Es vital para la sincronización del catálogo. Cuando el backend notifica un cambio (ej. venta exitosa), el frontend actúa como un observador reactivo. Al recibir el evento, actualiza el estado de las obras (deshabilitando botones, cambiando colores a gris, mostrando mensajes de "Agotado") de manera inmediata para el usuario.



4. Análisis de decisiones técnicas

4.1 ¿Por qué solo 1 patrón en el Frontend?

Se decidió limitar la implementación de patrones complejos en el frontend para evitar el "Over-engineering" (complejidad innecesaria). La arquitectura de React/Frontend moderno es altamente declarativa; implementar patrones como Strategy o State mediante clases pesadas en el navegador habría dificultado la gestión del DOM. El Observer se mantuvo por su naturaleza reactiva, esencial para la experiencia de usuario en tiempo real.

4.2 Cumplimiento de objetivos

Objetivo del Proyecto	Patrón aplicado	Estado
Integridad de Ventas	COMMAND	Implementado y funcional

Objetivo del Proyecto	Patrón aplicado	Estado
Escalabilidad de Precios	STRATEGY	Implementado y funcional
Gestión de Inventario	STATE	Implementado y funcional
Notificaciones Reactivas	OBSERVER	Implementado en Backend y Frontend